Self Project 2025-06-03

MCP 2.0

A polished MCP 2.0 post that uses normal markdown, guide-style sections, expressive code blocks, GitHub repo cards, tables, and checklists.

No cover image available.

Project Snapshot

DATE
2025-06-03

CATEGORY
Self Project

SHORT DESCRIPTION
A polished MCP 2.0 post that uses normal markdown, guide-style sections, expressive code blocks, GitHub repo cards, tables, and checklists.

TAGS / TECH STACK
MCP | Protocols | AI Infrastructure | gRPC | Protobuf | Agents | Demo

Project Links

GitHub Repository

Full Project Content

Project Repository Card

GitHub Repository: anubhaparashar/MCP2.0

Introduction

This post is designed to showcase the different kinds of content display supported by this blog theme in one place.

It combines:

- normal markdown writing
- guide-style technical explanation
- expressive code blocks
- a GitHub repository card
- tables and checklists

The project featured here is **MCP 2.0**, a concept that explores what a stronger, more production-ready Model Context Protocol could look like.

Why This Project Matters

As AI systems become more complex, protocol design starts to matter more.

MCP 2.0 explores how AI-to-tool communication can become:

- more strongly typed
- more stream-friendly
- easier to discover
- more secure
- more reusable across agents

> The main value of this project is not just implementation.
> It is the systems thinking behind the protocol design.

Guide-Style Walkthrough

What problem is it solving?

Traditional tool-calling and context-sharing approaches often become fragile at scale.

Common pain points include:

1. weak typing
2. awkward streaming support
3. no native service discovery
4. broad security scopes
5. custom integration glue for every workflow

What does MCP 2.0 propose?

MCP 2.0 pushes the protocol toward a more structured model with:

- typed RPC
- capability-based security
- middleware hooks
- discovery services
- multi-agent delegation
- event-driven interaction patterns

Markdown Elements Showcase

Checklist

- [x] Add frontmatter
- [x] Add GitHub repo card
- [x] Add expressive code block
- [x] Add architecture diagram
- [x] Add video embed
- [ ] Add custom cover image later

Table

Area	MCP Today	MCP 2.0 Direction
Typing	Looser	Stronger typed contracts
Streaming	Limited	First-class
Discovery	Manual	Dynamic
Security	Broad scopes	Capability-based
Agent chaining	Ad hoc	Protocol-aware

Quote

> Good protocol design reduces integration friction before it becomes a platform problem.

Proposed Architecture

```
LLM Client / Agent
        │
        ▼
Capability Token + Delegation Headers
        │
        ▼
MCP 2.0 Gateway / Middleware Layer
   ├── Auth & Policy Checks
   ├── Observability
   ├── Cache / Retry / Circuit Breakers
   └── Routing / Discovery
        │
        ▼
Core Services
   ├── Discovery Service
   ├── Context Tool Server
   ├── Event Bus Server
   └── Registry Server
        │
        ▼
Databases / APIs / Enterprise Tools / Other Agents
```

This view shows that MCP 2.0 is not only a transport idea.
It is trying to become a protocol layer for governed, scalable AI interactions.

Repository Structure

```
MCP2.0/
├── README.md
├── auth.py
├── client_example.py
├── context_tool_server.py
├── event_bus_server.py
├── init_db.sql
├── middleware.py
├── protos/
│   └── mcp2.proto
├── registry_server.py
└── requirements.txt
```

File Walkthrough

- **README.md** — explains the motivation and protocol vision
- **auth.py** — auth and access control concepts
- **middleware.py** — reusable cross-cutting protocol behavior

- **registry_server.py** — service registration and lookup
- **context_tool_server.py** — context and tool interaction layer
- **event_bus_server.py** — event-driven communication support
- **client_example.py** — client-side usage example
- **init_db.sql** — setup script
- **requirements.txt** — dependencies
- **protos/mcp2.proto** — typed protocol contract

Expressive Code Block Showcase

The blog theme styles fenced code blocks automatically.

```
syntax = "proto3";

package mcp2;

service Discovery {
  rpc Register(RegisterRequest) returns (RegisterResponse);
  rpc Lookup(LookupRequest) returns (LookupResponse);
}
```

And here is a Python example:

```
class CapabilityToken:
    def __init__(self, subject, allowed_methods):
        self.subject = subject
        self.allowed_methods = allowed_methods

    def can_call(self, method_name: str) -> bool:
        return method_name in self.allowed_methods


token = CapabilityToken("agent-a", ["lookup", "register"])
print(token.can_call("lookup"))
```

Design Goals

The main design goals of MCP 2.0 can be summarized as:

- **typed RPC**
- **native streaming**
- **dynamic discovery**
- **fine-grained capability security**

- **pluggable middleware**
- **multi-agent delegation**

This makes the project interesting not only as code, but as a protocol design exercise.

References

- Repository: `anubhaparashar/MCP2.0`
- Key files: `README.md`, `auth.py`, `client_example.py`, `context_tool_server.py`, `event_bus_server.py`, `init_db.sql`, `middleware.py`, `protos/mcp2.proto`, `registry_server.py`, `requirements.txt`